Engineering Lead Assessment - Take Home Assignment (Basic)

Scenario

AI Sales Doctor helps B2B companies find and qualify leads using AI, 3rd party databases and enrichment APIs.
Your task is to build a **simple Lead Management MVP** that:

1. Displays leads
2. Allows adding new leads
3. Supports basic filtering

Part A – Backend (25 min)

Build a simple API using **Python (FastAPI)**.
You may store data **in memory** (no database required).

Part B – Frontend (25 min)

Use any **React.js** framework.

UI Requirements

- ☐ [Page 1] List leads in a table with id, name, job title, phone number, company name, personal email, call_status, headcount and industry.
- ☐ [Page 2] Form to add new lead(s)
- ☐ Filter by industry and headcount
- ☐ Button to "Enrich" lead(s)

Engineering Lead Assessment - Take Home Assignment (Advanced), 1

Multi-Line Dialer (2 lines) + Update CRM (or Lead Management MVP)

Stack: Node.js (Express/Fastify) + React (Vite/Next)
AI tools allowed: Yes (Codex/Copilot/Cursor/Claude/etc.). Show what you verified.

Goal

Build a CRM where an agent runs a **2-line dialer session** over a list of leads. When a call ends, the system **writes a CRM activity record** for that lead (mock CRM).

Submission

Deliverables shall be hosted on any free webhost service, CRM tool and utilize any other free existing platform plans. Email the url, Git repo links to ellee@aisalesdr.co & eulee@aisalesdr.co

- Repo with instructions:
  - npm install
  - npm run dev (or separate client/server commands)
- NOTES.md:
  - tradeoffs, what you'd do next
  - how you used AI tools + what you verified

Engineering Lead Assessment - Take Home Assignment (Advanced), 2

Part 1- Backend (node.js)

Data Models (it is okay for this to be in memory)

- Lead
  - id, name, company, phone, email
  - crmExternalId (optional)
- Call Status
  - id, leadId, sessionId, status
  - startedAt, endedAt
  - Call_Status: CONNECTED | NO_ANSWER | BUSY | VOICEMAIL | CANCELED_BY_DIALER
  - providerCallId (string)
- DialerSession
  - id, agentId
  - leadQueue (array of leadIds)
  - concurrency fixed to 2
  - activeCallIds (max 2)
  - winnerCallId (nullable)
  - status: RUNNING | STOPPED
  - metrics: attempted, connected, failed, canceled
- CRMActivity
  - id, leadId, crmExternalId, type (e.g., CALL)
  - callId, disposition, notes, createdAt

Engineering Lead Assessment - Take Home Assignment (Advanced), 3

Part 2 – CRM Integration

Implement Mock CRM inside backend

These endpoints represent an external CRM system (store data in memory):

CRM Sync behavior (required)

When a call reaches a terminal outcome:

- If lead has no crmExternalId, update contact
- Then create activity with disposition + basic notes
- Save the activity in your own app DB (CRMActivity model) and the mock CRM store.

Idempotency requirement

- The same terminal call event should not create duplicate CRM activities. (Use a simple idempotency key like callId.)

API endpoints to inspect CRM results

- GET /mock-crm/contacts
- GET /mock-crm/activities
- GET /leads/:id/crm-activities (your app's view)

Part 3 – Frontend (REACT)

Screen 1: Leads + Session creation

- Display seeded leads list (4–8 leads)
- Select leads (checkboxes)
- "Create Dialer Session" + "Start"

Screen 2: Dialer Session

- Show 2 active "lines" (call cards):
  - lead name, phone, status
- Show session metrics
- Show winner call (if connected)
- Show CRM activity creation status (per call/lead)
- Poll backend every 1–2 seconds